

Middleware

1. Middleware層の責務

Middleware層は、HTTPリクエストがControllerへ届く前に、共通処理を行う。

認証確認、権限確認、CSRF確認、RateLimit、CORS、セキュリティヘッダー、request_id付与、アクセスログ、panic復旧など。

一方で、保存可否、評価可否、ランキング計算、モーダル表示可否、RefreshToken rotation、reuse検知、DB更新などの業務判断は行わない。

項目	Middlewareでやる	Middlewareでやらない
認証	AccessTokenを検証し、認証済みUser情報を渡す	Signup/Login/Refreshの業務処理はしない
権限	Admin権限が必要なルートを入力で弾く	Admin操作の監査ログ作成はUsecase側
CSRF	CSRF Cookie値とHeader値の一致確認	RefreshToken rotation、reuse検知はしない
GuestSession	未ログインユーザー識別情報を取得または作成し、渡す	Guestの行動集計や興味スコア計算はしない
Actor確認	UserまたはGuestSessionのどちらかが存在することを確認する	User/Guestの作成・取得処理はしない
RateLimit	APIごとの利用回数制限を行う	Redis Clientを直接触らない
CORS	許可Origin、Header、Methodを制御する	業務上のアクセス可否判断はしない
SecureHeaders	セキュリティ系HTTPヘッダーを付ける	HTML sanitizeはValidator / AdminUsecase側
Recover / Logger	panic復旧、アクセスログ、request_id付与	業務ログ・監査ログの代替にはしない

2. Middleware一覧

Middleware名	何をするか	呼び出すUsecase	主な対象	失敗時
<code>RecoverMiddleware</code>	panic時に500で返す	なし	全API	500
<code>RequestIDMiddleware</code>	request_idを発行し、後続処理とログに渡す	なし	全API	原則失敗なし
<code>AccessLogMiddleware</code>	request_id, method, path, status, latencyを記録する	なし	全API	原則失敗なし
<code>SecureHeadersMiddleware</code>	セキュリティ系HTTPヘッダーを付与する	なし	全API	原則失敗なし
<code>CORSMiddleware</code>	許可Origin/Header/Method/Credentialsを制御する	なし	全API	Browser側で断
<code>BodyLimitMiddleware</code>	膨大なRequest Bodyを拒否する	なし	POST/PUT/PATCH	413
<code>AuthMiddleware</code>	AccessTokenを検証し、UserID/Role/token_versionをAppContextへ保存する	<code>IAuthUsecase</code>	user/admin API	401/403
<code>OptionalAuthMiddleware</code>	TokenがあればUserとして扱い、なければGuest候補として通す	<code>IAuthUsecase</code>	public + personalization API	不正tokenありなら401
<code>AdminMiddleware</code>	AppContextのroleがadminか確認する	なし	admin API	403
<code>CSRFMiddleware</code>	CSRF CookieとHeaderを照合する	なし	refresh/logout	403
<code>GuestSessionMiddleware</code>	未ログイン時にGuestSessionを取得または作成する	<code>IGuestSessionUsecase</code>	guest/event/search/modal	500
<code>ActorMiddleware</code>	UserまたはGuestSessionのどちらかがあることを保証する	なし	event/recommendation/modal	401
<code>RateLimitMiddleware</code>	APIごとの利用回数制限を判定する	<code>IRateLimitUsecase</code>	auth/event/save/rating/admin	429/503

3. AppContext設計

Echoの `c.Set()` / `c.Get()` を各所で直接使うと、any や型アサーションが散らばる。
any型を使わないため、Middlewareで取得した共通情報は独自の AppContext に集約する。

AppContext は、認証済みUser情報、GuestSession情報、request_idなど、ControllerがUsecase inputへ変換するために必要な最小情報だけを持つ。

Controller / Usecase に渡すための最小限のリクエスト情報だけを保持する。

項目	型	説明	保存してよいか
RequestID	string	リクエスト追跡用	OK
AuthUserID	*uint64	認証済みUserID。未ログインならnil	OK
AuthRole	*entity.UserRole	user / admin。未ログインならnil	OK
TokenVersion	*int	AccessToken内のtoken_version。未ログインならnil	OK
GuestSessionID	*uint64	Guest識別ID。ログイン済みならnil	OK
ClientIPHash	string	RateLimit用のIP hash	OK
RawAccessToken	string	生AccessToken	保存しない
RawRefreshToken	string	生RefreshToken	保存しない
Password	string	入力password	保存しない

Actorのルール

状態	AuthUserID	GuestSessionID	扱い
ログイン済み	設定	nil	User Actor
ゲスト	nil	設定	Guest Actor
両方nil	nil	nil	Actorなし。ActorMiddlewareで401
両方あり	設定	設定	不正状態。ActorMiddlewareで401

ログイン済みUserの場合、GuestSessionMiddlewareはGuestSessionを作成・保存しない。

AppContextの方針

方針	内容
Middleware	AppContextに認証済み情報、Guest情報、request_idだけ詰める
Controller	AppContextからUserID/GuestSessionIDを取り出し、Usecase inputへ変換する
Usecase	AppContextもEcho Contextも受け取らない
Repository	Middlewareから直接呼ばない
生Token	AppContextに保存しない
Password	AppContextに保存しない

AppContext に保存してはいけない値

値	保存禁止の理由
RawAccessToken	漏洩時に認証突破されるため
RawRefreshToken	漏洩時に長期ログインを奪われるため
Password	漏洩時の被害が最大になるため
CSRF token raw value	ログやContext経由で漏洩する必要がないため
Cookie header raw value	RefreshToken等を含む可能性があるため

これらの値は、必要な処理の直前に読み取り、検証後は保持しない。
ログ出力も禁止する。

4. Middleware適用順

Middlewareの順番を間違えると、認証漏れ、CSRF漏れ、RateLimit漏れ、ログ欠落が起きる。

5. 全体共通Middleware

順番	Middleware	理由
1	<code>RecoverMiddleware</code>	panicを必ず拾い、500で返すため
2	<code>RequestIDMiddleware</code>	以降のログ・エラーにrequest_idを付けるため
3	<code>SecureHeadersMiddleware</code>	全レスポンスに安全ヘッダーを付けるため
4	<code>CORSMiddleware</code>	Frontend originを制限するため
5	<code>BodyLimitMiddleware</code>	膨大なRequest Bodyを早い段階で拒否するため
6	<code>AccessLogMiddleware</code>	request/responseのログを残すため

6. 認証が必要なUser API

保存、評価、保存一覧、自分の情報取得。

順番	Middleware	理由
1	<code>AuthMiddleware</code>	UserID/Role/token_versionを確定するため
2	<code>RateLimitMiddleware</code>	user_id単位で制限するため
3	<code>Controller</code>	Validator → Usecaseへ進む

7. Admin API

AdminのBean管理、Article管理、関連付け管理、手動バッチ、監査ログ確認。

順番	Middleware	理由
1	<code>AuthMiddleware</code>	認証済みUserを確定するため
2	<code>AdminMiddleware</code>	admin以外を403で弾くため
3	<code>RateLimitMiddleware</code>	admin操作も連打制限するため
4	<code>Controller</code>	Validator → AdminUsecaseへ進む

8. Refresh API

RefreshはAccessTokenが期限切れでも使うため、`AuthMiddleware` は使わない。

RefreshTokenの検証、期限確認、失効確認、reuse検知、rotationは `AuthUsecase` の責務。

順番	Middleware	理由
1	<code>RateLimitMiddleware</code>	CSRF攻撃や連打も先に制限するため
2	<code>CSRFMiddleware</code>	Cookie + Header一致を確認するため
3	<code>Controller</code>	CookieからRefreshTokenを取り、AuthUsecase.Refreshへ渡す

9. Logout API

Logoutは認証済みUserの操作なので、先に `AuthMiddleware` でUserを確定。

その後、user_id単位でRateLimitを行い、最後にCSRFを確認。

順番	Middleware	理由
1	<code>AuthMiddleware</code>	認証済みUserを確定するため
2	<code>RateLimitMiddleware</code>	user_id単位でlogout連打を制限するため
3	<code>CSRFMiddleware</code>	Cookie + Header一致を確認するため
4	<code>Controller</code>	AuthUsecase.Logoutへ進む

10. Guest / Event / Recommendation / Modal API

ログイン済みならUserとして扱う。未ログインならGuestSessionを作成または取得。

順番	Middleware	理由
1	OptionalAuthMiddleware	TokenがあればUserとして扱うため
2	GuestSessionMiddleware	未ログインならGuestSessionを確定するため
3	ActorMiddleware	UserまたはGuestSessionのどちらかを保証するため
4	RateLimitMiddleware	actor単位で制限するため
5	Controller	Validator → Usecaseへ進む

11. AuthMiddleware設計

AuthMiddleware は、Authorization HeaderのAccessTokenを検証し、認証済みUser情報を AppContext に保存。

AccessToken検証では、署名、期限、claims、token_version、User statusを確認。

順番	処理	何をしているか	呼び出し先	失敗時
1	Authorization Header確認	Bearer <token> があるか確認	なし	401
2	token形式確認	Bearer形式以外を拒否	なし	401
3	AccessToken検証	署名、期限、claimsを確認	AuthUsecase.ValidateAccessToken	401
4	token_version確認	token内tvとDB側token_versionを比較	ValidateAccessToken 内	401
5	User status確認	active以外を拒否	ValidateAccessToken 内	403
6	AppContext保存	user_id, role, token_versionを保存	なし	なし
7	次へ進む	Controllerへ渡す	next	なし

11.1. AuthMiddlewareが直接やらないこと

NG	理由
UserRepositoryを直接呼ぶ	MiddlewareがRepository依存になる
RefreshTokenを扱う	AccessToken認証とRefresh処理が混ざる
CookieからAccessTokenを読む	CSRFリスクが上がる。AccessTokenはAuthorization Header前提
user_idをBodyから読む	なりすまし可能
token_version比較をMiddleware内に直接書く	認証業務がMiddlewareに漏れる
logout処理を書く	LogoutはAuthUsecaseの責務

12. OptionalAuthMiddleware設計

OptionalAuthMiddleware は、AccessTokenがあればUserとして扱い、なければGuestとして後ろへ通す。

ただし、Authorization Headerが存在するのにtokenが不正な場合は401で止める。

状態	処理
Authorization Headerなし	Guest候補として次へ進む
Authorization Headerあり、token正常	AppContextにUser情報を保存して次へ進む
Authorization Headerあり、token不正	401
Authorization Headerあり、期限切れ	401

OptionalAuthMiddleware は、Public API、Recommendation、Modal、Eventなど、ログイン済みでもGuestでも使えるAPIに適用する。

13. AdminMiddleware設計

`AdminMiddleware` は、`AuthMiddleware` の後に実行。

AppContextに保存されたroleを確認し、admin以外を403で拒否。

AdminUsecase側でも管理者前提の業務判断は持つが、入口で弾くためにMiddlewareでも確認。

順番	処理	何をしているか	失敗時
1	Auth確認	AppContextにAuthUserがあるか確認	401
2	role確認	roleがadminか確認	403
3	次へ進む	Admin Controllerへ渡す	なし

14. GuestSessionMiddleware設計

`GuestSessionMiddleware` は、未ログインユーザーの行動ログを紐付けるために使う。

ログイン済みUserの場合はGuestSession作成をスキップ。

GuestSessionの生値はCookieに保存し、DBにはhashのみ保存。

順番	処理	何をしているか	呼び出し先	失敗時
1	Auth確認	既にUserならGuestSession作成をスキップ	なし	なし
2	Cookie確認	<code>guest_session_key</code> cookieを読む	なし	なし
3	GetOrCreate	有効なら取得、なければ新規作成	<code>GuestSessionUsecase.GetOrCreate</code>	DB障害なら500
4	Cookie再設定	新規・期限切れ・不正時はcookieを更新	なし	500
5	AppContext保存	<code>guest_session_id</code> を保存	なし	なし
6	次へ進む	Controllerへ渡す	<code>next</code>	なし

14.1. GuestSessionの扱い

状態	対応
<code>guest_session_key</code> がない	新規作成
<code>guest_session_key</code> が期限切れ	古いcookieを削除し、新規作成
<code>guest_session_key</code> が不正形式	古いcookieを削除し、新規作成
DB障害	500
Cookie設定失敗	500

14.2 Guest Cookie設定

Cookie	値	HttpOnly	Secure	SameSite	Path
<code>guest_session_key</code>	<code>__Host-guest_session</code>	<code>true</code>	本番 <code>true</code>	<code>Lax</code>	<code>/</code>

本番でFrontendとBackendが同一siteの場合、GuestSession Cookieは SameSite=Lax を優先する。

本番でFrontendとBackendが別siteの場合、GuestSession Cookieは SameSite=None; Secure を使用する。

AllowCredentials: true を使用する場合、AllowOrigins: * は禁止する。

`refresh_token` と `csrf_token` のCookie仕様は「18. CSRF Cookie設定」に集約する。

15. CSRFMiddleware設計

`CSRFMiddleware` は、Cookieを使う更新系エンドポイントに適用。

RefreshTokenはHttpOnly Cookieで扱うため、`POST /auth/refresh` と `POST /auth/logout` にはCSRF対策が必要。

方式はDouble Submit Cookieとする。

順番	処理	何をしているか	失敗時
1	Method確認	GET/HEAD/OPTIONSは原則対象外	なし
2	CSRF Cookie取得	<code>csrf_token</code> を読む	403
3	Header取得	<code>X-CSRF-Token</code> を読む	403
4	一致確認	Cookie値とHeader値を比較	403
5	次へ進む	Controllerへ渡す	なし

16. ActorMiddleware設計

ActorMiddleware は、Event、Recommendation、Modal、SearchなどのAPIで、行動主体が存在することを保証する。ここでいうActorとは、ログイン済みUserまたはGuestSessionのどちらか一方である。

ActorMiddleware はUsecaseを呼ばない。

前段の OptionalAuthMiddleware と GuestSessionMiddleware がAppContextに保存した値を確認するだけ。

順番	処理	何をしているか	失敗時
1	User確認	AppContextにAuthUserIDがあるか確認	なし
2	Guest確認	AppContextにGuestSessionIDがあるか確認	なし
3	actor判定	UserIDまたはGuestSessionIDの片方だけ存在するか確認	両方nilなら401
4	二重actor拒否	UserIDとGuestSessionIDが両方ある状態を拒否	401
5	次へ進む	Controllerへ渡す	なし

ActorMiddlewareがやらないこと

NG	理由
AuthUsecaseを呼ぶ	認証処理はOptionalAuthMiddlewareで完了しているため
GuestSessionUsecaseを呼ぶ	GuestSession取得はGuestSessionMiddlewareで完了しているため
DB存在確認をする	Middlewareの責務ではないため
行動ログを作成する	EventUsecase / SearchUsecase / ModalUsecaseの責務であるため
UserIDとGuestSessionIDの両方を許可する	行動主体が二重になり、DB制約とも矛盾するため

17. CSRF対象

API	CSRF	理由
<code>GET /auth/csrf</code>	不要	CSRF token発行用
<code>POST /auth/signup</code>	不要	認証Cookie前提ではない
<code>POST /auth/login</code>	不要	認証Cookie前提ではない
<code>POST /auth/refresh</code>	必須	RefreshToken Cookieを使う
<code>POST /auth/logout</code>	必須	RefreshToken失効・Cookie削除を伴う
<code>POST /auth/logout-all</code>	必須	全RefreshToken失効・Cookie削除を伴う
<code>GET /search/beans</code>	不要	参照系GET。DB更新やre_search記録は行わない
<code>GET /search/articles</code>	不要	参照系GET。DB更新やre_search記録は行わない
<code>POST /events</code>	必須	GuestSession Cookieで行動記録するため。re_searchもここで記録する
<code>POST /recommendation-modal/:id/click</code>	必須	GuestSession Cookieで状態更新するため
<code>POST /recommendation-modal/:id/close</code>	必須	GuestSession Cookieで状態更新するため
<code>POST /saved</code>	原則不要	Authorization Header認証前提
<code>DELETE /saved</code>	原則不要	Authorization Header認証前提
<code>POST /ratings</code>	原則不要	Authorization Header認証前提
<code>POST /admin/*</code>	原則不要	Authorization Header + AdminMiddleware前提

18. CSRF Cookie設定

Cookie	値	HttpOnly	Secure	SameSite	Path
refresh_token	RefreshToken生値	true	本番true	Lax or Strict	/auth
csrf_token	ランダム値	false	本番true	Lax or Strict	/

19. RateLimitMiddleware設計

RateLimitMiddleware は、APIごとの利用回数制限を行う。

ただし、MiddlewareはRedis Clientを直接扱わない。

MiddlewareはRateLimit判定に必要な情報を集め、RateLimitUsecase に渡す。RateLimitUsecase がルール選択とTokenBucket判定を行い、RateLimitRepository がRedis/Luaで残数・補充・消費を処理。

項目	内容
Middleware	key生成に必要な情報を集める
RateLimitUsecase	ルール選択、TokenBucket判定、結果返却
RateLimitRepository	Redis/Luaで残数・補充・消費を処理
Controller	RateLimit処理は書かない

19.1. RateLimitの重要方針

Signup/Loginのemail単位RateLimitは、Middlewareでbodyを読まず、AuthUsecase側で行う。

理由は、MiddlewareでRequest Bodyを読むと、Controllerが後でbodyを読めなくなる可能性があるため。

対象	Middlewareでやる	Usecaseでやる
Signup	IP単位のRateLimit	email正規化後にemail単位RateLimit
Login	IP単位のRateLimit	email正規化後にemail単位RateLimit
Refresh	IP単位のRateLimit	RefreshToken検証・reuse検知
Event	actor単位のRateLimit	イベント記録可否
Save	user_id単位のRateLimit	保存可否
Rating	user_id単位のRateLimit	評価可否
Admin Batch	admin_user_id + job_name単位	手動実行可否

19.2. RateLimit対象

対象	key	制限例	失敗時
Signup	ip_hash	5回 / 10分	429
Signup email	normalized_email	AuthUsecase側で制限	429
Login	ip_hash	5回 / 10分	429
Login email	normalized_email	AuthUsecase側で制限	429
Refresh	ip_hash	30回 / 10分	429
Event記録	actor_id + event_type	60回 / 分	429
Search	actor_id	30回 / 分	429
Recommendation	actor_id	20回 / 分	429
Save	user_id	30回 / 分	429
Rating	user_id	30回 / 分	429
Modal	actor_id	20回 / 分	429
Admin CRUD	admin_user_id	60回 / 分	429
Admin Batch	admin_user_id + job_name	3回 / 時	429

19.3. RateLimitMiddleware処理手順

順番	処理	何をしているか	呼び出し先	失敗時
1	ルール名決定	routeごとに login, event, admin_batch などを決める	なし	500
2	key材料取得	IP hash、user_id、guest_session_idなどを取得	なし	400/401
3	RateLimit判定	TokenBucketで許可/拒否を判定	RateLimitUsecase.Allow	429/503
4	Header付与	remaining, retry_after等を返す	なし	なし
5	次へ進む	Controllerへ渡す	next	なし

19.4. RateLimit Header

Header	内容
X-RateLimit-Name	適用されたルール名
X-RateLimit-Remaining	残りtoken数
Retry-After	429時、再試行までの秒数
X-Request-ID	問い合わせ・ログ追跡用

20. SecureHeadersMiddleware設計

SecureHeadersMiddleware は、基本的なセキュリティ系HTTPヘッダーを付与。

Header	値	理由
X-Content-Type-Options	nosniff	MIME type誤認防止
X-Frame-Options	DENY	clickjacking防止
Referrer-Policy	strict-origin-when-cross-origin	Referer漏洩を抑える
Permissions-Policy	geolocation=(), microphone=(), camera=()	不要なブラウザ機能を無効化
Cache-Control	認証系のみ no-store	token系レスポンスのキャッシュ防止

20.1. CSPについて

FrontendではCSPを設定し、不要なinline scriptを許可しない。

Backend APIでは、認証・token系レスポンスに Cache-Control: no-store を付与する。

Header	担当	方針
Content-Security-Policy	Frontend中心	inline script禁止、画像・API接続先を制限
X-Content-Type-Options	Backend	nosniff
X-Frame-Options	Backend/Frontend	DENY
Cache-Control	Backend	認証系レスポンスは no-store

21. CORS設計

FrontendからBackend APIへ通信するため、CORSを設定。

項目	値
AllowOrigins	local: http://localhost:3000、本番: FEドメイン
AllowMethods	GET, POST, PUT, PATCH, DELETE, OPTIONS
AllowHeaders	Authorization, Content-Type, X-CSRF-Token, X-Request-ID
AllowCredentials	true
禁止	AllowOrigins: * + AllowCredentials: true

AllowCredentials: true を使う場合、AllowOrigins: * は禁止。

理由は、Cookie付き通信を任意のOriginから許可してしまうため。

22. API別Middleware適用表

API分類	Middleware
Public Bean一覧/詳細	RateLimitPublicRead
Public Article一覧	RateLimitPublicRead
Article Preview	RateLimitPublicRead
Article Full Detail	AuthMiddleware → RateLimitUser
Signup	RateLimitAuthIP → Controller → AuthUsecase側でEmail RateLimit
Login	RateLimitAuthIP → Controller → AuthUsecase側でEmail RateLimit
CSRF発行	RateLimitPublicRead
Refresh	RateLimitRefresh → CSRFMiddleware
Logout	AuthMiddleware → RateLimitUser → CSRFMiddleware
Search	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitSearch
Event記録	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitEvent → CSRFMiddleware
Save	AuthMiddleware → RateLimitUser
Rating	AuthMiddleware → RateLimitUser
Recommendation	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitRecommendation
Modal表示	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitModal
Modalクリック/閉じる	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitModal → CSRFMiddleware
Admin Bean	AuthMiddleware → AdminMiddleware → RateLimitAdmin
Admin Article	AuthMiddleware → AdminMiddleware → RateLimitAdmin
Admin Relation	AuthMiddleware → AdminMiddleware → RateLimitAdmin
Admin Batch	AuthMiddleware → AdminMiddleware → RateLimitAdminBatch
Admin Audit	AuthMiddleware → AdminMiddleware → RateLimitAdmin

23. Middleware Interface設計

Middlewareは、Usecaseをinterface経由で呼び出す。

MiddlewareからRepositoryやRedis Clientを直接呼ばない。

Usecaseを呼ぶ可能性があるinterfaceには、必ず `context.Context` を渡す。

理由は、リクエストキャンセルやタイムアウトをDB/Redis処理へ伝えるため。

```

type IAuthUsecase interface {
    ValidateAccessToken(ctx context.Context, rawToken string) (AuthUser, error)
}

type IGuestSessionUsecase interface {
    GetOrCreate(ctx context.Context, rawSessionKey string) (GuestSessionResult, error)
}

type IRateLimitUsecase interface {
    Allow(ctx context.Context, input RateLimitInput) (RateLimitResult, error)
}

type ILogger interface {
    Info(ctx context.Context, input LogInput)
    Error(ctx context.Context, input LogInput)
}

```

23.1. Middleware用DTO

DTO	フィールド	説明
AuthUser	UserID uint64, Role entity.UserRole, TokenVersion int	認証済みユーザー
GuestSessionResult	GuestSessionID uint64, RawSessionKey string, Created bool	GuestSession結果
RateLimitInput	Name string, Key string, Cost int	RateLimit判定入力
RateLimitResult	Allowed bool, Remaining int, RetryAfterSec int	RateLimit判定結果
LogInput	RequestID, Method, Path, Status, LatencyMs, UserID, GuestSessionID, ErrorCode	アクセスログ入力

24. エラー変換

UsecaseはHTTP statusを返さない。

ただし、Middleware内でリクエストを止める場合は、MiddlewareがHTTP responseを返す。

Middleware内のエラーは、共通エラーレスポンス形式に変換する。

Middleware	エラー	HTTP
AuthMiddleware	Headerなし	401
AuthMiddleware	token不正・期限切れ	401
AuthMiddleware	token_version不一致	401
AuthMiddleware	user停止中	403
AdminMiddleware	admin以外	403
CSRFMiddleware	csrf tokenなし	403
CSRFMiddleware	csrf不一致	403
ActorMiddleware	User/GuestSessionなし	401
GuestSessionMiddleware	DB障害	500
RateLimitMiddleware	上限超過	429
RateLimitMiddleware	Redis障害	auth/write/adminは503
BodyLimitMiddleware	body過大	413
RecoverMiddleware	panic	500

25. やってはいけないこと

NG	理由
MiddlewareからGORMを直接呼ぶ	Clean Architectureが崩れる
MiddlewareからRedis Clientを直接呼ぶ	RateLimitRepositoryの責務を奪う
MiddlewareからRepositoryを直接呼ぶ	Usecaseを飛ばして責務が崩れる
Refresh処理をMiddlewareに書く	reuse検知・rotationはAuthUsecaseの責務
user_idをRequest Bodyから信用する	なりすまし可能
guest_session_idをRequest Bodyから信用する	他人のGuestとして行動できる
AccessTokenをCookie保存前提にする	CSRFリスクが上がる
CSRFを全APIに雑に適用する	Authorization Header APIには不要で、実装が複雑化する
signup/loginでCSRF必須にする	認証Cookie前提ではないため優先度が低い
RateLimitなしでlogin/refresh/eventを開ける	brute force・spam・集計汚染の原因
Admin BatchをRateLimitなしにする	手動連打で集計・DB負荷が壊れる
RateLimitMiddlewareでRequest Bodyを読む	Controllerでbodyを読めなくなる事故が起きやすい
ActorMiddlewareでUsecaseを呼ぶ	前段Middlewareとの責務が重複する
Middlewareにランキング計算を書く	RankingUsecaseの責務を奪う

NG	理由
Middlewareにモーダル表示可否を書く	ModalUsecaseの責務を奪う
Middlewareに保存可否・評価可否を書く	SaveUsecase / RatingUsecaseの責務を奪う